



FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE DE
COIMBRA

Projeto de Engenharia e Ciência de Dados

Predicting Student Outcomes from GitLab Activity

André Cunha 2020242388
Lucas Saraiva 2021218251
Matas Bazilevicius 2024167528
Tiago Carlos 2021218666

Contents

1 Introduction 2

2 Data Collection and Preparation 2

2.1 GitLab Data Retrieval 2

2.2 Feature Extraction 2

2.3 User Normalization and Matching 3

2.4 Data Cleaning and Integration 3

3 Exploratory Data Analysis (EDA) 3

4 Machine Learning Model 3

4.1 Problem Definition and Architecture 3

4.2 Training and Evaluation 4

5 Web Application 4

5.1 System Overview 4

5.2 Key Features and Pages 5

5.3 Data Flow and Visualization 5

6 Challenges and Design Decisions 5

7 Prerequisites, Setup, and Running the Application 6

7.1 Prerequisites 6

7.2 Project Setup 7

7.3 Running the Application 7

8 Conclusion and Future Work 8

1 Introduction

This project aims to predict student outcomes (grades) in a software engineering course based on their GitLab repository activity. Our team (Lucas Saraiva, Tiago Carlos, Matas Bazilevicius, and André Cunha) collected GitLab data, preprocessed it into features, trained a regression model, and built a Flask web app for demonstrations. Prior research has shown that patterns of source-control usage can indicate student success , motivating our approach. In summary, we fetched commits, merge requests, and issues from each course repository via the GitLab API, extracted relevant metrics (counts and code changes), trained a PyTorch neural network to predict final grades, and exposed this functionality through a web interface for instructors to use.

2 Data Collection and Preparation

2.1 GitLab Data Retrieval

Data was obtained using the `python-gitlab` library to query each project's activity. Specifically, we listed all commits, merge requests (MRs), and issues for the student project repository in GitLab.

2.2 Feature Extraction

From these records we extracted metrics for each student, including:

- Commit metrics: number of commits, total lines added, total lines deleted, and cumulative lines changed. Commit timestamps were recorded to compute time-based features.
- Merge Request metrics: number of MRs created by the student, and number of MRs merged into the main branch. For each MR we also stored its creation and merge dates.
- Issue metrics: number of issues opened, number of issues assigned to the student, and number of issues closed. We recorded issue creation and close dates, as well as any discussion notes count (if available).
- Date tracking: lists of dates for each event type (commit dates, MR creation/merge dates, issue open/ close dates) were saved for temporal analysis.

2.3 User Normalization and Matching

All activity was aggregated per student. We normalized GitLab usernames and names by lowercasing and removing special characters so that each student's contributions could be correctly linked (using both `author_name` and `author_email` from commits, and `author` fields from MRs/issues). This ensured that e.g. "John.Doe" in GitLab matched the same student in our grade list. The output of this step was a table (one row per student) with all the above feature counts and lists of dates, which we exported to CSV for analysis.

2.4 Data Cleaning and Integration

We performed exploratory data analysis (EDA) to understand and clean the dataset. First, we merged the GitLab-derived features with the students' grade CSV by matching normalized GitLab username to student records. We checked for missing values: students with no activity on a metric had that feature set to zero. We also verified that the distribution of each feature (e.g. number of commits, lines changed) was reasonable and did not contain outliers that would break the model. GitLab activity was mapped to individual students by using a lookup: for each commit or issue author we normalized their name/email and matched it to the student's username.

3 Exploratory Data Analysis (EDA)

Data cleaning steps included: combining the per-repository dictionaries into a single DataFrame, converting list-valued columns (e.g. all commits) into aggregated counts, and removing any duplicate or irrelevant entries. We ensured every student had exactly one record with their total activity counts and final grade. In summary, each student record now contained the normalized user handle, team name, total counts for commits/MRs/issues (as above), and their final grade from the course. This cleaned dataset was then split into training and test sets for modeling.

4 Machine Learning Model

4.1 Problem Definition and Architecture

We posed grade prediction as a regression task (predicting a score from 0 to 20). Inspired by the project guidelines, we implemented a PyTorch neural network to perform regression. Our final model takes nine input features (merges created, merges merged, lines added, lines deleted, lines

changed, commits, issues created, issues assigned, issues closed). The network architecture is a simple multi-layer perceptron: one hidden layer of size 64 with ReLU activation, followed by a linear output unit (see code in SimpleNN or MyModel).

4.2 Training and Evaluation

We trained the model using mean-squared-error (MSE) loss on the training split (80% of data) and held out 20% for validation.

Prior to training, each feature was standardized to zero mean and unit variance (using the means and standard deviations from the training set). This normalization step is important for neural networks, as it ensures all inputs are on comparable scales, which typically improves convergence and performance. At inference time, the same stored means and stds are applied to new data before feeding it to the network. The model outputs a continuous value; we round this to the nearest integer and clamp it to the 0–20 range to produce the final predicted grade. In our experiments, the trained model achieved a test MSE of around 6.35 (with example predicted vs actual values printed during evaluation), indicating it captured some of the grade variance. (For brevity, we did not implement full evaluation metrics like R^2 or MAE, but this could be added.)

5 Web Application

5.1 System Overview

The trained model and data processing were integrated into a Flask web application to allow interactive use. The app uses the Flask-Login extension for user session management. Professors and students can register and log in; the registration form records each user's email, password, and occupation (professor or student). Professors are then prompted to enter their GitLab personal access token and to fetch their project repositories. Using this token, the app calls the GitLab API to list repositories the professor has access to.

Once a repository is selected, the app fetches and stores all student activity metrics from that repo using the same API methods as above (`fetch_gitlab_commits` , `fetch_gitlab_issues` , `fetch_gitlab_merge_requests`). Student records (matching usernames) are created in the SQLite database with fields for each feature.

5.2 Key Features and Pages

- Repository Selection: A form where professors input the GitLab token and choose a repository from a dropdown .
- Student List: After selecting a repository, the app displays a table of students (one per row) with their current activity metrics.
- Predictions: A page where the professor can trigger the model. They can select a single student or “Predict All.” For each prediction, the code gathers the student’s nine feature values, normalizes them (subtract mean, divide by std), runs them through the loaded PyTorch model, The page then writes the resulting integer grade back to the database (clamped to 0–20) then shows each student’s updated predicted grade.
- Weekly History Comparison: A page to compare a student’s weekly activity. When a student is selected, the app collects their stored date lists (commits, MRs created/merged, issues opened/ closed) and converts them into weekly counts. It provides a JSON endpoint that returns these counts per ISO week, which are then plotted on a multi-line chart . This allows professors to see how a student’s activity evolved week by week relative to midterm or project deadlines.

5.3 Data Flow and Visualization

Throughout the app, visual elements like charts are generated by JavaScript fetching data from Flask endpoints . The Flask-Login system ensures only authenticated users access these pages, and that professors can only see data for their selected repository. The model and normalization tensors are pre-loaded from disk when the app starts, so predictions occur in real time without retraining.

6 Challenges and Design Decisions

During development, several challenges and design trade-offs were encountered:

Data consistency and name mapping: GitLab users often differed from the official student list. We normalized strings by lowercasing and stripping punctuation/accents to match names and emails to usernames. We built a mapping from author names/email prefixes to GitLab usernames so all commits and issues could be attributed correctly.

Missing or sparse data: Some students had very little activity (or none), which meant many feature values were zero. We decided to keep these zeros rather than drop students, under the

assumption that lack of activity itself is informative. The model learned to handle these cases, although it does bias predictions for low-activity students. Model complexity and accuracy: We chose a simple two-layer neural net to avoid overfitting on our limited data. This meant accuracy was moderate. For example, test predictions tended to hover around the mean grade with some variance.

Real-time requirements: To support live predictions without delay, the model is pre-trained and loaded at startup. The feature normalization tensors (mean, std) were also saved so that new inputs can be standardized exactly as during training. This design means the app does not need to re-fit the model on each query; it simply runs inference. Tool choices: We used SQLite for the database for simplicity and portability. The Flask framework (with Flask-Login and Flask-WTF) provided quick development of secure user forms and routes. For visualization, we used Matplotlib to test plots in analysis, and Chart.js in the web UI for dynamic charts.

7 Prerequisites, Setup, and Running the Application

To make the project fully reproducible, this section explains the required libraries, how to configure the environment, and how to run the Flask-based web application locally.

7.1 Prerequisites

Make sure you have the following installed:

- Python 3.8 or higher
- Git
- Virtualenv (optional, but recommended)

Install the necessary packages by running:

```
pip install flask flask_sqlalchemy flask_login flask_wtf wtforms flask_bcrypt  
itsdangerous flask_mail python-gitlab torch matplotlib numpy pandas
```

These are the core dependencies used across the application, as listed in the `Install these libraries.md` file.

7.2 Project Setup

To run the application locally, you must begin by cloning the project repository and navigating to its root directory. Although not strictly necessary, it is recommended to create and activate a virtual environment to ensure dependency isolation. This can be done using the `venv` module included in Python; on Unix-based systems, the virtual environment is activated with source:

```
venv/bin/activate
```

while on Windows it uses:

```
venv\Scripts\activate
```

Once the environment is set, the required Python libraries must be installed. These include Flask and its main extensions (`flask_sqlalchemy`, `flask_login`, `flask_wtf`, `flask_bcrypt`, `flask_mail`), as well as data processing and machine learning libraries such as `python-gitlab`, `torch`, `numpy`, `pandas`, and `matplotlib`. All dependencies are listed in the supplementary file `Install these libraries.md` and can be installed using `pip`.

The application uses a SQLite database, which is already provided as `database.db`. However, if a fresh setup is preferred, the database schema can be re-initialized using Flask-Migrate commands: `flask db init`, followed by `flask db migrate` and `flask db upgrade`. This recreates the database structure based on the SQLAlchemy models defined in the code.

Finally, before running the application, it is important to ensure that the trained PyTorch model file (`model_new2.pth`) is placed in the correct directory. By default, the application loads this file from the `Code/` folder. If stored elsewhere, the path in the source code (specifically in `app.py`) must be updated accordingly. This model contains the saved weights used to perform student grade predictions in real time.

7.3 Running the Application

Start the Flask server with:

```
python app.py
```

This will launch the application on `http://127.0.0.1:5000/`.

8 Conclusion and Future Work

In summary, we successfully built an end-to-end system for predicting student grades from GitLab activity. The final deliverable lets an instructor authenticate with GitLab, fetch student repository data, compute activity metrics, and see grade predictions and activity charts in a user-friendly web interface.

Possible future improvements include:

- **Model tuning:** Experiment with more advanced models and systematic hyperparameter search. Gathering more data would also improve training and validation.
- **Enhanced analytics:** Add visualizations like scatter plots of features vs grades, correlation heatmaps, or assignment-by-assignment performance trends. More interactive dashboards could help educators explore the data.
- **Platform support:** Extend the tool to integrate with GitHub or other VCS platforms using their APIs, broadening applicability.
- **Usability features:** Implement email notifications to students (using Flask-Mail) when their predicted grade changes significantly, or integrate with LMS for alerts.

Overall, the framework is reproducible: given a GitLab token and project, one can rerun the data collection and retrain or reuse the model. The code and report together should allow others to replicate our pipeline or adapt it to similar educational data science projects.